Proyecto 1 — Cluster Kubernetes autoscalable para una aplicación CPU-bound

Objetivo: Desplegar una aplicación ficticia CPU-intensiva en Kubernetes con escalado automático (HPA/VPA) y demostrar comportamiento bajo carga.
Alcance: cluster k8s en cloud educativa; app (p. ej. worker que calcula primes o hashes); HPA configurado; métricas expositoras; pruebas de carga y gráficos.
Herramientas: Kubernetes (EKS/GKE/AKS o k8s en VM), kubectl, Metrics Server / Prometheus + Grafana (opcional), Horizontal Pod Autoscaler.
Arquitectura: CI repo → imagen Docker → despliegue en k8s → Service + HPA → Load generator.
Cronograma (fechas): H1 1–4 Nov: diseño, Dockerfile, cluster provisioning; H2 5–11 Nov: desplegar app y HPA; H3 12–18 Nov: pruebas de carga (k6/locust), medir replicas, latencias; H4 19–23 Nov: informe, capturas, demo.
Demo mínimo: mostrar despliegue, generar carga y evidenciar escalado automático (pods aumentan/disminuyen) y métricas.
Entregables: README, manifests, scripts, informe IEEE, slides, link demo.

Proyecto 2 — Pipeline CI/CD para microservicios con Kubernetes + GitHub Actions

Objetivo: Implementar pipeline CI/CD que construya imágenes, ejecute tests y despliegue automatizado a un cluster en cloud.
Alcance: 2 microservicios simples (API + worker), GitHub Actions workflows (build/test/push/deploy), rollback mínimo, despliegue en k8s.
Herramientas: GitHub Actions, Docker Hub / Container Registry, Kubernetes (EKS/GKE/AKS), Helm (opcional).
Arquitectura: Repo mono / multi → GitHub Actions → Registry → k8s (deployments/helm).
Cronograma: H1 diseño pipelines + estructura repos; H2 CI: build & test & push; H3 CD: manifests/Helm + pruebas de despliegue; H4 documentación + demo.
Demo mínimo: commit que dispara workflow, build y despliegue automático en cluster, mostrar histórico de pipelines y app funcionando.
Entregables: workflows, Dockerfiles, manifests, guía paso a paso, artículo IEEE.

Proyecto 3 — Servicio distribuido GPU-enabled para aceleración (batch jobs)

Objetivo: Desplegar un servicio capaz de ejecutar jobs que usan GPU (training/inferencia o kernel CUDA) en instancias con GPU y orquestar colas de jobs.
Alcance: crear worker que use GPU, cola de trabajos, scheduler simple, despliegue en nodos con GPU (o simulación con instancias educativas), medir tiempos GPU vs CPU.
Herramientas: Docker (nvidia-runtime si aplica), Kubernetes con taints/tolerations o nodos GPU, RabbitMQ/Redis como cola, framework pequeño (PyTorch/Numba/CUDA).
Arquitectura: Ingreso jobs → Broker → Scheduler → Workers GPU → resultados.

Cronograma: H1 diseño + setup GPU nodes; H2 containerizar y desplegar workers + broker; H3 ejecutar batch jobs y recopilar tiempos; H4 informe y demo.
Demo mínimo: enviar job y mostrar ejecución sobre GPU (tiempos) y resultado, además de métricas.
Entregables: imagen docker GPU, scripts, resultados comparativos, informe IEEE.

Proyecto 4 — Comparador de rendimiento: VM vs Container vs Serverless para función paralela

Objetivo: Diseñar el mismo trabajo paralelo (ej. procesamiento de N tareas) y ejecutar en 3 entornos: VM (instancia), contenedor en k8s y función serverless; comparar latencia, coste aproximado y escalado.
Alcance: implementar la tarea, orquestar ejecuciones, capturar métricas de tiempo/throughput/costo estimado.
Herramientas: Cloud VMs, Kubernetes, AWS Lambda / Azure Functions / Cloud Functions, herramientas de medición (time, logs).
Arquitectura: 3 entornos independientes con la misma workload generator.
Cronograma: H1 diseño de la carga y métricas; H2 implementación en cada entorno; H3 pruebas comparativas; H4 análisis, tablas y reporte.
Demo mínimo: ejecutar las 3 variantes y mostrar resultados comparados en tablas/gráficas.
Entregables: scripts, comandos reproducibles, informe con tablas y discusión.

Proyecto 5 — Sistema de cache distribuida y coherencia para tareas paralelas

Objetivo: Implementar un cache distribuido (Redis Cluster / Memcached) usado por múltiples workers paralelos y demostrar comportamiento (hit/miss, coherencia, rendimiento).
Alcance: cluster de Redis, conjunto de workers (Docker/K8s) que consultan/actualizan cache y base de datos persistente simple; medir latencia y throughput.
Herramientas: Redis Cluster, Docker/K8s, pequeña DB (Postgres), script supervisor.
Arquitectura: Clients → LoadBalancer → Workers → Redis Cluster ↔ DB.
Cronograma: H1 diseño y infra; H2 desplegar Redis + workers; H3 pruebas carga y coherencia; H4 documentación y demo.
Demo mínimo: mostrar workload, métricas de cache (hit ratio) y comparación con no-cache.
Entregables: configuraciones Redis, scripts, informe.

Proyecto 6 — Framework de mensajería y procesamiento paralelo con Kafka

Objetivo: Desplegar pipeline de streaming con Apache Kafka y consumidores paralelos que procesan eventos en paralelo (ej. transformación/filtrado). Medir throughput y latencia.
Alcance: cluster Kafka (con Zookeeper o KRaft), productores, múltiples consumidores en grupos, monitorización básica.
Herramientas: Kafka (con Strimzi o managed), Docker, consumidores en Python/Java, Prometheus/Grafana para métricas (opcional).
Arquitectura: Prod → Kafka topics → Consumer Groups (paralelismo) → sink (DB/file).
Cronograma: H1 diseño topics/partitions; H2 desplegar Kafka + producers/consumers; H3 test de throughput y tuning (partitions, replication); H4 reporte y demo.
Demo mínimo: enviar eventos alta tasa y mostrar consumidores paralelos y métricas de throughput/lag.
Entregables: scripts para levantar cluster, productores, consumidores, informe con gráficos.

Proyecto 7 — Implementación y evaluación de un scheduler distribuido simple

Objetivo: Diseñar e implementar un scheduler distribuido (master + workers) que asigne tareas considerando carga y capacidad, y evaluar su rendimiento frente a un scheduler centralizado.
Alcance: prototipo con políticas (round-robin, least-loaded), métricas de latencia y fairness, escenario con fallos de nodo para probar resiliencia.
Herramientas: Python/Go, Docker, Redis/Consul para estado (opcional).
Arquitectura: Scheduler(s) ↔ Workers ↔ Job queue; mecanismo de heartbeat y recuperación.
Cronograma: H1 diseño y API; H2 implementación y despliegue; H3 pruebas de escalado y fallos; H4 documentación.
Demo mínimo: mostrar asignación dinámica, métricas de latencia y comportamiento con fallos/reasignación.
Entregables: código scheduler, scripts de pruebas, informe

Proyecto 8 — Map-Reduce distribuido (mini) para procesamiento de texto

Objetivo: Implementar un framework simple tipo MapReduce (coordinador + mappers + reducers) para contar palabras en corpus grande y evaluar escalabilidad.
Alcance: diseñar jobs map/reduce, distribuir tareas en workers, recolectar resultados parciales, manejar fallas/retries.
Herramientas: Python/Java, Docker, storage en cloud (buckets), orquestación con k8s o VMs.
Arquitectura: Input (object storage) → Master → Mappers → Reducers → Output.

Cronograma: H1 diseño del flujo y dataset; H2 implementar mapping/reducing y despliegue; H3 pruebas con datasets de distinto tamaño; H4 análisis y reporte.
Demo mínimo: correr job sobre dataset grande y mostrar speedup al aumentar workers.
Entregables: código MapReduce, dataset de prueba, informe con análisis.

Proyecto 9 — Checkpointing y recuperación para jobs largos en cluster

Objetivo: Implementar checkpointing para un job largo (simulación numérica), permitir reinicio desde checkpoint tras fallo y medir overhead vs reaprovechamiento.
Alcance: job paralelo (MPI o multi-process), mecanismo de checkpoint (dump periódicos a storage), prueba de fallo y recuperación.
Herramientas: MPI / multiproceso, object storage (cloud bucket), scripts de orquestación.
Arquitectura: Job → periodic checkpoint → storage; upon failure → restart from last checkpoint.
Cronograma: H1 diseño del punto de control y formato; H2 implementación y storage; H3 tests de fallo y recuperación; H4 informe.
Demo mínimo: simular fallo y mostrar reinicio desde checkpoint con correctitud del resultado y tiempo/overhead medido.
Entregables: código, scripts de checkpoint/restart, informe.

Proyecto 10 — Scheduler y autoscaling basado en métricas personalizadas (custom metrics)

Objetivo: Implementar autoscaling no trivial donde las decisiones se tomen con métricas custom (p. ej. backlog queue length + latencia almacenamiento) y aplicar políticas para escalar horizontalmente.
Alcance: instrumentar app para exponer métricas custom, configurar Prometheus Adapter / custom metrics API, HPA basado en métricas personalizadas o KEDA.
Herramientas: Kubernetes, Prometheus, Prometheus Adapter / KEDA, exporter custom, load generator.
Arquitectura: App instrumented → Prometheus → Adapter/KEDA → HPA/KEDA → scaled pods.
Cronograma: H1 diseño métricas y endpoint; H2 instrumentación y Prometheus; H3 configurar autoscaler y pruebas; H4 documentar y demo.
Demo mínimo: generar workload que dispare escalado basado en metricas custom y mostrar comportamiento y métricas en dashboards.
Entregables: manifests, exporter code, scripts de prueba, informe y slides.

ARTICULO
El articulo debe contener al menos las siguientes secciones. Por cada seccion se describe el contenido esperado.
• Titulo adecuado
• Abstract: contiene las ideas mas importantes
• Cuerpo del art.culo: presenta un estilo legible y personal. No hay copy- paste e incluye lo siguiente:

o Descripcion del problema a solucionar
o Descripcion de las tecnologias utilizadas o Descripcion de la solucion
o Pruebas realizadas
• Conclusiones: resumen las ideas principales del articulo
• Bibliografia: Con el formato adecuado y referenciada en el texto
• Formato: segun las normas dadas: tablas y figuras referenciadas y numeradas, estilo y cuerpo de letra, jerarqu.a de los apartados y sub- apartados
• Formato IEEE, doble columna
• Maximo 10 paginas

PRESENTACIÓN
• Realiza una buena introducci.n del tema
• La presentacion es organizada y se puede seguir con facilidad
• Conocimiento y preparacion del tema
• Resumen apropiado de los puntos principales y presentacion de las conclusiones es bueno
• Se cumplen los objetivos o propositos enunciados en la presentacion
• La presentacion es interesante y amena
• Uso efectivo de ayudas audiovisuales
• Cumple con el tiempo asignado (no se extiende ni es demasiado breve)
• Tiempo M.ximo: 20 minutos

GUIA PASO A PASO
• Es completa e incluye toda la informacion necesaria para reproducir la solucion
• Es clara y se puede seguir facilmente Su presentacion es buena